

Contents

01 - 入门	3
安装	3
项目初始化	3
第一个程序	4
注释的写法	4
02 - 变量与类型	4
变量声明	4
使用 @const 声明不可变变量	4
类型标注	5
基本类型	5
字符串操作	6
转义序列	7
F-String (字符串插值)	7
类型转换 (as)	7
重新赋值规则	7
元组解构	8
03 - 运算符	8
算术运算符	9
比较运算符	9
逻辑运算符	10
位运算符	10
复合赋值运算符	10
自增 / 自减运算符	11
类型提升规则	11
成员运算符	12
控制流	12
if / else	12
case	13
while 循环	14
for 循环与 range	14
break 与 continue	15
嵌套示例	16
作用域规则	16
模式匹配	16
函数	18
基本函数定义	18
函数调用	18
递归函数	18
函数重载	18
省略返回类型 (Unit 类型)	19
默认参数	19
Lambda 函数	19
闭包	20
高阶函数	20
UFCS (统一函数调用语法)	21
练习	21
Record 与枚举	22
定义 Record	22

创建实例	22
字段访问（点记法）	22
字段赋值	22
Record 作为函数参数	23
嵌套 Record	23
枚举（enum）	23
带关联数据的枚举（ADT）	24
泛型枚举	25
运算符重载	25
练习	26
集合与迭代器	26
元组	26
列表	27
映射	29
集合	30
迭代器	31
练习	32
错误处理	32
Option 类型	33
Result 类型	33
契约式设计	35
使用场景选择	36
常见错误	36
练习	36
包	37
from/import 语法	37
子目录（点分隔路径）	37
目录包	37
相对导入	37
标准库（std）	38
搜索路径优先级	38
RY_PATH 环境变量	38
限制	38
并发	39
async/await	39
@parallel	40
OS 线程	40
网络（TCP 套接字）	41
选择合适的模型	43
常见错误	43
练习	43
测试	43
运行测试	43
编写测试	44
匹配器	44
输出格式	45
模拟（Mock）	45
参数化测试	46
基于属性的测试	46
使用契约进行测试	46

限制	47
练习	47
构建项目	47
项目设置	48
步骤 1：定义数据模型	48
步骤 2：任务操作	48
步骤 3：显示	49
步骤 4：主程序	49
步骤 5：编写测试	50
下一步	51
English 日本語 简体中文	

01 - 入门

下一篇教程 -> [02 - 变量与类型](#)

安装

快速安装 (macOS Apple Silicon)

```
curl -fsSL https://raw.githubusercontent.com/t0k0shi/ry/main/install.sh | sh
```

ry 二进制文件将安装到 ~/.local/bin，标准库将安装到 ~/.ry/share/std/。

请确保 ~/.local/bin 已添加到 PATH：

```
export PATH="$HOME/.local/bin:$PATH"
```

如需从源代码构建或在其他平台安装，请参阅 [README 的安装部分](#)。

项目初始化

使用 ry new 命令创建新项目：

```
ry new my-project
```

```
cd my-project
```

这将生成以下文件和目录：

- package.toml - 项目配置文件
- src/main.ry - 入口点（附带示例代码）

若要将当前目录初始化为项目，请使用 ry init：

```
mkdir my-project
```

```
cd my-project
```

```
ry init
```

详情请参阅[项目管理](#)。

第一个程序

将以下内容保存为 `hello.ry` 文件：

```
print("Hello, World!")
```

使用以下命令运行：

```
ry hello.ry
```

输出：

```
Hello, World!
```

也可以通过管道或 Here-document 使用 `-c` 标志从标准输入运行代码：

```
echo 'print("Hello, World!")' | ry -c
```

```
ry -c <<'RY'
print("Hello, World!")
RY
```

注释的写法

从 `#` 到行尾的内容会被视为注释。

```
# 这是一段注释
print("Hello") # 也可以在行尾添加注释
```

注释不会影响代码的执行。

下一篇教程 -> [02 - 变量与类型](#)

[English](#) | [日本語](#) | [简体中文](#)

02 - 变量与类型

[<- 01 - 入门](#) / [下一篇-> 03 - 运算符](#)

变量声明

在 Ry 中，使用简单的赋值语法声明变量。默认情况下，变量是可变的。

```
x = 42          # 推断为 int
y = 3.14        # 推断为 float
flag = true     # 推断为 bool
name = "Ry"     # 推断为 str
```

使用 `@const` 声明不可变变量

`@const` 指令将变量标记为不可变（常量）。声明后无法更改其值。

```
@const
x = 42          # 推断为 int
```

```

@const
y = 3.14      # 推断为 float

@const
flag = true   # 推断为 bool

@const
name = "Ry"   # 推断为 str

```

类型标注

可以显式指定变量的类型。

```

x: int = 42

rate: float = 0.5

ok: bool = false

msg: str = "hello"

```

当类型标注与实际值的类型不一致时，会产生编译错误。

基本类型

类型	说明	字面值示例
int	64 位整数	0, 42, -10
u8	无符号 8 位整数 (0-255)	b: u8 = 42
float	64 位浮点数	0.0, 3.14, -1.5
bool	布尔值	true, false
str	字符串	"hello", ""

底层数值类型

Ry 还提供底层数值类型，用于精确控制内存布局。这些类型没有隐式转换——必须使用 `as` 进行显式转换。

类型	说明	示例
i8	8 位有符号整数	x: i8 = 42
i16	16 位有符号整数	x: i16 = 100
i32	32 位有符号整数	x: i32 = 42
i64	64 位有符号整数	x: i64 = 100
u8	8 位无符号整数	x: u8 = 200
u16	16 位无符号整数	x: u16 = 60000
u32	32 位无符号整数	x: u32 = 3000000000
u64	64 位无符号整数	x: u64 = 100
f32	32 位浮点数	x: f32 = 3.14

```

a: i32 = 10
b: i32 = 20
c = a + b      # OK: i32 + i32 → i32

```

```
d = 42
# e = a + d      # Error: cannot mix i32 and int
```

```
y = a as int      # 显式转换为 int
z = d as i32      # 显式转换为 i32
```

```
# 无符号类型使用无符号运算
x: u32 = 3000000000
y: u32 = 7
q = x / y          # 无符号除法 (UDiv)
```

注意: 底层整数的 / 执行整数除法 (类似 Rust), 而非浮点除法。有符号类型使用 SDiv, 无符号类型使用 UDiv。

注意: 底层整数的算术运算在溢出时会回绕。有符号类型使用二进制补码, 无符号类型使用模运算。如果担心溢出, 请使用高级 int 类型 (64 位)。

固定长度数组

对于底层类型, Ry 提供固定长度连续数组 T[N]。这些数组在栈上分配, 大小在编译时确定。

```
buf: i32[4] = [1, 2, 3, 4]
print(buf[0])      # 1
buf[2] = 99
print(buf[2])      # 99
print(length(buf)) # 4

pixels: u8[3] = [255, 128, 0]
```

字符串操作

字符串支持多种操作。

```
a = "Hello"
b = "World"

# 拼接
c = a + ", " + b    # "Hello, World"

# 比较 (字典序)
print(a == b)      # false
print(a != b)      # true
print(a < b)       # true ("H" < "W")

# 长度
print(length(a))   # 5

# 子字符串检查
s = "Hello, World!"
print(contains(s, "World"))    # true
print(starts_with(s, "Hello")) # true
print(ends_with(s, "!"))      # true
```

转义序列

字符串中可使用以下转义序列。

序列	含义
<code>\n</code>	换行
<code>\r</code>	回车
<code>\t</code>	Tab
<code>\\</code>	反斜杠
<code>\"</code>	双引号
<code>\0</code>	空字符

```
print("Hello\nWorld")  # 分成两行输出
print("A\tB")          # 以 Tab 分隔
print("say \"hi\"")     # 包含双引号的字符串
```

F-String (字符串插值)

当需要在字符串中嵌入变量或表达式时，使用 `f"..."` 代替 `+` 拼接。`{}` 中的表达式会被自动求值并转换为字符串。

```
name = "Alice"
print(f"Hello {name}")  # Hello Alice

x = 3
y = 4
print(f"{x} + {y} = {x + y}")  # 3 + 4 = 7
```

使用 `{` 和 `}` 输出字面花括号：

```
print(f"{{escaped}}")  # {escaped}
```

为什么使用 **f-string**？它们比手动拼接 `"Hello " + name` 更具可读性且更不容易出错。当混合文本和动态值时，请使用 **f-string**。

常见错误：不加 `f` 前缀写 `"Hello {name}"` 会得到字面文本 `{name}`，而非变量的值。

类型转换 (as)

使用 `as` 进行显式类型转换。窄化转换（如 `float` 到 `int`）以及数值类型与非数值类型（`str`、`bool`）之间的转换需要 `as`，而某些安全的数值提升（如表达式和某些调用中的 `int` 到 `float`）会隐式发生。

```
x = 42 as float      # 42.0
y = 3.14 as int       # 3 (向零截断)
s = 42 as str         # "42"
b = true as int       # 1
```

常见错误：将 `float` 转换为 `int` 是向零截断的，所以 `3.9 as int` 得到 3 而非 4。如需四舍五入，请使用 `math` 模块的 `round()`。

重新赋值规则

未使用 `@const` 声明的变量可以重新赋值，但有以下限制：

```

x = 10
x = 20      # OK: 重新赋予相同类型的值
# x = "text" # 错误: 禁止更改类型的重新赋值

@const 变量无法重新赋值。

@const
N = 5
# N = 6 # 错误: 禁止对 @const 变量重新赋值

也无法重新声明同名的变量。

x = 1
# 同一作用域内禁止重新声明同名变量

```

元组解构

可以在单次声明中将元组拆解为多个变量。

```

@const
a, b = (10, 20)
print(a)  # 10
print(b)  # 20

```

通配符

使用 `_` 忽略特定位置的值。

```

@const
x, _ = (1, 2)  # 只绑定 x; 2 被丢弃
print(x)      # 1

```

可变变量解构

省略 `@const` 即可声明可变变量。

```

a, b = (10, 20)
a = 99
print(a)  # 99

```

规则

- 左侧的变量数量必须与元组的元素数量相符。
 - 每个变量遵循与普通声明相同的 `@const`/可变规则。
 - 不支持嵌套元组解构。
-

[<- 01 - 入门 / 下一篇-> 03 - 运算符](#)

[English](#) | [日本語](#) | [简体中文](#)

03 - 运算符

[<- 02 - 变量与类型 / 下一篇-> 04 - 控制流](#)

算术运算符

运算符	说明	示例	结果
+	加法	3 + 2	5
-	减法	3 - 2	1
*	乘法 / 字符串重复	3 * 2	6
/	除法 (始终为 float)	7 / 2	3.5
//	整除 (int 操作数为 int，任一为 float 则为 float)	7 // 2	3
%	取模	7 % 3	1
**	幂运算 (始终为 float)	2 ** 10	1024

```
a = 10
b = 3
```

```
print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256 (float)
```

溢出安全：int 的算术运算（+、-、*、一元-）如果结果超出 64 位有符号范围，会引发运行时错误。溢出的常量表达式在编译时即被捕获。底层类型（i32、u8 等）会静默回绕——如需显式溢出控制，请使用 checked_add/saturating_add/wrapping_add。

比较运算符

所有比较运算符返回 bool 值。

运算符	说明	示例
==	等于	a == b
!=	不等于	a != b
<	小于	a < b
<=	小于等于	a <= b
>	大于	a > b
>=	大于等于	a >= b

```
x = 5
y = 10
```

```
print(x == y)    # false
print(x != y)    # true
print(x < y)     # true
print(x <= y)    # true
print(x > y)     # false
print(x >= y)    # false
```

比较运算符也适用于字符串（字典序比较）。

```
print("abc" == "abc")    # true
print("abc" < "abd")     # true
print("b" > "a")         # true
```

逻辑运算符

运算符	说明	示例
and	逻辑 AND	a and b
or	逻辑 OR	a or b
not	逻辑 NOT	not a

```
t = true
f = false

print(t and f)    # false
print(t or f)     # true
print(not t)      # false
print(not f)      # true
```

位运算符

位运算符仅适用于 int 类型。

运算符	说明	示例
&	位与	5 & 3 -> 1
	位或	5 3 -> 7
^	位异或	5 ^ 3 -> 6
~	位取反（一元）	~5 -> -6
<<	左移	1 << 3 -> 8
>>	算术右移	8 >> 2 -> 2
>>>	逻辑右移	-1 >>> 1 -> 9223372036854775807

```
a = 0b1010    # 10
b = 0b1100    # 12

print(a & b)    # 8 (0b1000)
print(a | b)    # 14 (0b1110)
print(a ^ b)    # 6 (0b0110)
print(~a)       # -11
print(1 << 4)   # 16
print(32 >> 2)  # 8
```

复合赋值运算符

更新变量值时可使用的简写语法。

运算符	说明	等价表达式
+=	加法赋值	x = x + n
-=	减法赋值	x = x - n
*=	乘法赋值	x = x * n
/=	除法赋值	x = x / n
%=	取模赋值	x = x % n
//=	整除赋值	x = x // n
**=	幂运算赋值	x = x ** n
&=	位与赋值	x = x & n
=	位或赋值	x = x n
^=	位异或赋值	x = x ^ n
<<=	左移赋值	x = x << n
>>=	右移赋值	x = x >> n

```
x = 10
x += 5    # x == 15
x -= 3    # x == 12
x *= 2    # x == 24
x /= 4    # x == 6 (变为 float)
```

自增 / 自减运算符

将变量增减 1 的简写语法。

运算符	说明	等价表达式
x++	加 1	x = x + 1
x--	减 1	x = x - 1

```
count = 0
count++    # count == 1
count++    # count == 2
count--    # count == 1
```

注意：仅可作为语句使用，不能在表达式中使用。

类型提升规则

以下说明运算中 int 与 float 混合时的行为。

```
# + - * 当其中一方为 float 时结果为 float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3 (float)
print(1.0 + 2)    # 3 (float)

# / 始终为 float
print(4 / 2)      # 2 (float)

# // 若两侧皆为 int 则为 int，任一为 float 则为 float
print(7 // 2)     # 3 (int)
print(7.0 // 2)   # 3 (float)
```

```

# ** 始终为 float
print(2 ** 3)      # 8 (float)

# % 当两边皆为 int 时为 int，其中一方为 float 时为 float
print(7 % 3)       # 1 (int)
print(7.5 % 2)     # 1.5 (float)

# + 当两边皆为 str 时为字符串拼接
print("foo" + "bar") # "foobar"

# * 当一方为 str、另一方为 int 时为字符串重复
print("ab" * 3)     # "ababab"
print(3 * "ab")     # "ababab"

```

成员运算符

运算符	说明	示例
in	成员检查	2 in {1, 2, 3} -> true
not in	否定成员检查	4 not in {1, 2, 3} -> true

```

s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true

```

[<- 02 - 变量与类型](#) / [下一篇-> 04 - 控制流](#)

[English](#) | [日本語](#) | [简体中文](#)

控制流

[<- 上一篇：运算符](#) | [下一篇：函数 ->](#)

if / else

使用 if 根据条件进行分支处理。

```
x = 10
```

```

if x > 0:
    print(x)
else:
    print(0)

```

- else 可以省略。
- 条件不限于 bool 值。对于 int，0 视为假，非 0 视为真。
- if 语句可以嵌套。

```

a = 5
b = 3

```

```

if a > 0:
    if b > 0:
        print(a + b)    # 8

```

case

使用 `case`: 进行无主题的多分支条件判断，或使用 `case value`: 进行带主题的模式匹配。这两种形式取代了旧的 `when` 与 `match` 关键字。

条件分支 `case`:

```

x = -2

case:
    x > 0:
        print("positive")
    x < 0:
        print("negative")
    -:
        print("zero")

```

当需要链接多个分支时，这是推荐的形式。

模式匹配 `case value`:

```

enum Color:
    Red
    Green
    Blue

c = Color::Green
case c:
    Color::Red:
        print("red")
    Color::Green:
        print("green")
    Color::Blue:
        print("blue")

```

使用此形式安全地解构 `enum`、`Option`、`Result` 和字面值模式。

case: 表达式

`case`: 也可以用作表达式。`_ =>` 通配符分支是必需的。

```

label = case:
    score >= 90 => "A"
    score >= 80 => "B"
    _ => "C"

```

这替代了嵌套的三元表达式，使多分支值选择更具可读性。

case value: 表达式

`case value`: 也可以用作表达式，使用 `=>` 从每个分支返回值。

```

res = case x:
    Some(v) => v
    None    => 0

label = case direction:
    Direction::North => "N"
    Direction::South => "S"
    Direction::East  => "E"
    Direction::West  => "W"

category = case score:
    n if n >= 90 => "A"
    n if n >= 80 => "B"
    -           => "F"

```

case 表达式支援与 case 语句相同的所有模式：字面值、变量、enum、Option、Result、OR 模式 (|)、守卫 (if) 和通配符 (_)。case 必须是穷尽的。

if 表达式

对于只需要两个分支并产生值的条件判断，可使用 if 表达式：

```

abs_val = if x >= 0 => x else -x
label = if score >= 90 => "A" else "B"

```

else 分支是必需的，且两个分支必须产生相同类型的值。对于多分支表达式，请改用 case:。

while 循环

当条件为真时，重复执行块。

```

i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1

```

for 循环与 range

可使用列表或 range 进行迭代。

```

for x in [1, 2, 3]:
    print(x)
# 1
# 2
# 3

```

range(n) 产生从 0 到 n - 1 的整数。

```

for i in range(5):
    print(i)
# 0
# 1

```

```
# 2
# 3
# 4
```

`range(start, end)` 产生从 `start` 到 `end - 1` 的整数。

```
for i in range(2, 5):
    print(i)
# 2
# 3
# 4
```

`..` 范围运算符创建包含两端的范围：`1 .. 3` 产生 `[1, 2, 3]`。

```
for i in 1 .. 3:
    print(i)
# 1
# 2
# 3
```

使用 `for k, v in map` 可以遍历映射的键值对：

```
m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)
```

break 与 continue

`break` 立即跳出循环。`continue` 跳过当前迭代，进入下一次迭代。

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3
```

在 `while` 循环中也可同样使用。

```
n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
    print(n)
# 1
# 3
# 5
# 7
```

注意：在嵌套循环中，`break` / `continue` 仅作用于最内层的循环。在循环外使用会产生编译错误。

嵌套示例

for 和 while 循环可以嵌套。

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)

# 11
# 13
# 21
# 23
# 31
# 33
```

作用域规则

控制流块有自己的作用域。

块作用域

在块内声明的变量无法从块外部引用。

```
if true:
    inner = 42
# 在此处引用 inner 会产生编译错误
```

引用与重新赋值外部变量

可以从块内引用和重新赋值外部的变量。

```
count = 0
for i in range(5):
    count = count + i
print(count)    # 10
```

内层作用域的重新赋值

在块内对变量赋值会修改外层的变量（Python 风格的作用域）。不会产生遮蔽——内层的赋值会修改同一个变量。

```
x = 1
if true:
    x = 99
    print(x)    # 99
print(x)        # 99
```

模式匹配

case value: 安全地对 enum、Option、Result 和字面值进行分支。

```
enum Color:
    Red
    Green
    Blue
```



```

c = Color::Green
case c:
  Color::Red:
    print("red")
  Color::Green:
    print("green")
  Color::Blue:
    print("blue")
# green

```

Option 匹配

使用 case value: 安全地处理 Some 和 None 两种情况。

```

x: Option<int> = Some(42)
case x:
  Some(v):
    print(v)
  None:
    print("nothing")
# 42

```

通配符与字面值

_ 是匹配任何值的通配符模式。也可以使用字面值（数字、字符串、布尔值）进行匹配。

```

n = 5
case n:
  0:
    print("zero")
  1:
    print("one")
  _:
    print("other")
# other

```

守卫子句

可以使用 if 添加守卫条件。

```

case n:
  x if x > 0:
    print("positive")
  x if x < 0:
    print("negative")
  _:
    print("zero")

```

注意: case value: 必须是穷尽的。对于 enum，必须覆盖所有变体。对于 Option，需要同时覆盖 Some 和 None。对于字面值，需要 _ 通配符。

[<- 上一篇：运算符](#) | [下一篇：函数 ->](#)

[English](#) | [日本語](#) | [简体中文](#)

函数

[<- 上一篇：控制流](#) | [下一篇：Record 与枚举 ->](#)

基本函数定义

函数使用 `function` 关键字定义。参数类型以 `name: type` 格式指定。如果省略类型，默认为 `any`。返回类型在 `->` 之后指定。

```
function add(a: int, b: int) -> int:
    return a + b
```

- 建议声明参数类型。如果省略，类型默认为 `any`。
 - 返回类型在 `->` 之后指定。
 - 使用 `return` 返回值。
-

函数调用

通过名称和参数调用已定义的函数。

```
function multiply(x: int, y: int) -> int:
    return x * y

result = multiply(3, 4)
print(result)    # 12
```

递归函数

函数可以调用自身（递归）。

```
function factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120
print(factorial(0))    # 1
```

函数重载

可以定义多个同名但参数数量或类型不同的函数。

```
function add(a: int, b: int) -> int:
    return a + b

function add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))        # 3
print(add(1.5, 2.5))    # 4
```

调用时会根据参数类型自动选择适当的函数。

注意：定义参数类型相同但仅返回类型不同的函数会产生编译错误。

省略返回类型 (Unit 类型)

不需要返回值的函数可以省略 `->`。此时函数返回 `Unit` 类型。

```
function greet():  
    print(42)
```

```
greet()    # 42
```

这是最简单的无参数、无返回值函数形式。

默认参数

参数可以有默认值。调用者省略这些参数时，将使用默认值。

```
function greet(name: str, greeting: str = "Hello") -> str:  
    return f"{greeting}, {name}"
```

```
print(greet("Alice"))           # Hello, Alice  
print(greet("Bob", "Good morning")) # Good morning, Bob
```

可以有多个默认参数：

```
function connect(host: str, port: int = 8080, timeout: int = 30) -> str:  
    return f"{host}:{port} (timeout={timeout})"
```

```
print(connect("localhost"))           # localhost:8080 (timeout=30)  
print(connect("localhost", 3000))     # localhost:3000 (timeout=30)  
print(connect("localhost", 3000, 10)) # localhost:3000 (timeout=10)
```

为什么使用默认参数？它们让你在常见情况下保持简洁的调用方式，同时在需要时允许自定义——无需多个重载。

注意：有默认值的参数必须放在无默认值的参数之后。

Lambda 函数

Lambda 函数允许你将函数写成表达式。单表达式 lambda 使用 `(parameters) => expression` 形式，块 lambda 使用 `(parameters):` 后跟缩进块。两种情况下返回类型都会自动推断。

单表达式 Lambda

```
double = (x: int) => x * 2  
print(double(5))    # 10
```

```
add = (a: int, b: int) => a + b  
print(add(3, 4))    # 7
```

无参数 Lambda

```
answer = () => 42  
print(answer())    # 42
```

多行 Lambda

在 `:` 之后换行并缩进，可以编写多条语句。

```
abs = (x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3))  # 3
```

为什么使用 `lambda`？它们非常适合简短的一次性函数——特别是作为 `filter` 和 `map` 等高阶函数的参数（见下文）。

闭包

Lambda 函数可以捕获定义它们的作用域中的变量。函数及其捕获的环境的组合称为闭包。

```
offset = 10
add_offset = (x: int) => x + offset
print(add_offset(5)) # 15
```

闭包按值捕获变量——创建闭包后修改原始变量不会影响闭包的副本。

```
base = 10
f = (x: int) => x + base
base = 999
print(f(1)) # 11（仍然使用捕获的值 10）
```

这是双向的——闭包内部的修改也不会影响外部变量：

```
counter = 0
items = [1, 2, 3]
items.map((x: int):
    counter += x # 只修改闭包的本地副本
    return x
)
print(counter) # 0（外部变量不变）
```

为什么按值捕获？它确保了安全性和可预测性——你总是可以只看当前作用域就能推断变量的值，无需担心闭包内部发生的修改。为什么使用闭包？它们让你可以即时创建专用函数。例如，你可以从单个模板创建一系列加法函数。

高阶函数

可以定义接受其他函数作为参数的函数。函数类型写作 `function(parameter_types) -> return_type`。

```
function apply(f: function(int) -> int, x: int) -> int:
    return f(x)

double = (x: int) => x * 2
print(apply(double, 3)) # 6
print(apply((n: int) => n + 1, 10)) # 11
```

函数作为值

命名函数也可以绑定到变量或作为参数传递 —— 它们的行为与 `lambda` 完全相同。

```
function square(x: int) -> int:
    return x * x
```

```
# 将命名函数作为参数传递
print(apply(square, 4)) # 16
```

```
# 绑定到变量
sq = square
print(sq(5)) # 25
```

为什么使用高阶函数？它们让你将做什么与怎么做分离。同一个 `apply` 函数可以与任何变换一起使用，使代码更具复用性。你已经在[集合](#)中看到过这种模式，如 `filter`、`map` 和 `reduce`。

UFCS（统一函数调用语法）

使用 UFCS，你可以将 `f(a, b)` 写成 `a.f(b)`。第一个参数移到点号前面，实现方法链式调用风格。

```
function add(a: int, b: int) -> int:
    return a + b
```

```
x = 1
print(x.add(2)) # add(x, 2) -> 3
```

链式调用

UFCS 在链接多个调用时特别出色 —— 从左到右阅读而不是从内到外：

```
function double(n: int) -> int:
    return n * 2
```

```
# 链式（自然阅读：“取 x，加 2，然后翻倍”）
print(x.add(2).double()) # 6
```

```
# 等价的嵌套调用（更难阅读）
print(double(add(x, 2))) # 6
```

为什么使用 UFCS？它将深层嵌套的函数调用转变为可读的从左到右的管道。你已经在[迭代器链](#)中见过这种用法，如 `xs.iter().filter(...).map(...).to_list()`。

练习

1. 默认参数：编写函数 `format_price(amount: int, currency: str = "USD", decimals: int = 2) -> str`，用于格式化价格。验证 `format_price(42)` 和 `format_price(42, "EUR")` 都能正常工作。
 2. 高阶函数：编写函数 `apply_twice(f: function(int) -> int, x: int) -> int`，将 `f` 应用于 `x` 两次（即 `f(f(x))`）。使用 `(x: int) => x + 1` 测试，验证 `apply_twice((x: int) => x + 1, 5)` 返回 7。
 3. UFCS 链式调用：定义 `inc(n: int) -> int`（加 1）和 `triple(n: int) -> int`（乘以 3）。使用 UFCS 编写 `5.inc().triple()` 并验证结果为 18。
-

[<- 上一篇：控制流 | 下一篇：Record 与枚举 ->](#)

[English](#) | [日本語](#) | [简体中文](#)

Record 与枚举

[<- 上一篇：函数 | 下一篇：集合与迭代器 ->](#)

定义 Record

使用 `record` 关键字定义 `record`。各字段以 `name: type` 格式描述。

```
record Point:
    x: int
    y: int
```

Record 是在栈上分配的值类型。

创建实例

像调用函数一样使用 `record` 名称来创建实例。参数按照字段的定义顺序指定。

```
p = Point(10, 20)
```

字段访问（点记法）

使用点记法访问字段。

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

Record 可以直接打印：

```
p = Point(10, 20)
print(p)      # Point(x: 10, y: 20)
```

字段赋值

未使用 `@const` 声明的可变变量的字段可以重新赋值。

```
p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

注意：对 `@const` 声明的变量的字段赋值会产生编译错误。

Record 作为函数参数

可以将 record 作为函数的参数传递。

```
record Point:
  x: int
  y: int

function distance_x(a: Point, b: Point) -> int:
  return a.x - b.x

p1 = Point(10, 3)
p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

嵌套 Record

Record 的字段可以使用其他 record。

```
record Point:
  x: int
  y: int

record Line:
  start: Point
  end: Point

line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

通过链接点记法可访问嵌套字段。

枚举 (enum)

使用 `enum` 关键字定义枚举。每个变体作为具名常量处理。

定义

```
enum Color:
  Red
  Green
  Blue
```

使用方式

使用 `::` 访问变体。

```
c = Color::Red
print(c)    # Red
```

比较

可以使用 `==` 和 `!=` 比较变体。

```
case:
  c == Color::Red:
    print("red!")
  c == Color::Green:
    print("green!")
  _:
    print("blue!")
```

函数参数

可以使用 enum 名称作为函数的参数类型。

```
function describe(c: Color) -> str:
  if c == Color::Red:
    return "warm"
  return "cool"
```

带关联数据的枚举 (ADT)

枚举变体可以携带关联值。这让单个枚举可以表示一系列不同形状的数据——这种模式称为代数数据类型 (ADT)。

```
enum Shape:
  Circle(radius: float)
  Rectangle(width: float, height: float)
  Point
```

命名字段仅用于文档目的——使定义具有自描述性。无名语法 (Circle(float)) 同样有效。

构造 ADT 变体

构造始终是位置性的，无论字段是否命名。

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

匹配 ADT 变体

使用 case 从 ADT 变体中提取关联数据。绑定使用你选择的变量名，而非字段名。这直接与你在[控制流](#)中学到的模式匹配相连接。

```
function describe(s: Shape) -> str:
  case s:
    Shape::Circle(r):
      return f"circle with radius {r}"
    Shape::Rectangle(w, h):
      return f"rectangle {w}x{h}"
    Shape::Point:
      return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0
```

为什么使用 ADT？它们让你以类型安全的方式建模“多种形状之一”的数据。编译器在模式匹配时确保你处理了每个变体，在编译时捕获遗漏的情况。

泛型枚举

枚举可以接受类型参数，使其可在不同的载荷类型间复用。

```
enum MyOption<T>:  
    MySome(T)  
    MyNone
```

使用方式

```
a = MyOption<int>::MySome(42)  
b: MyOption<int> = MyOption<int>::MyNone
```

```
case a:  
    MyOption::MySome(v):  
        print(v)          # 42  
    MyOption::MyNone:  
        print("none")
```

注意：Ry 的内置 `Option<T>` 和 `Result<T, E>` 类型的工作方式与此完全相同。你将在[错误处理](#)中学习它们。

运算符重载

可以使用 `function operator` 语法为自定义类型定义运算符。这让你的 `record` 可以自然地与 `+`、`==` 等运算符一起使用。

二元运算符

接受两个参数。

```
record Vec2:  
    x: int  
    y: int  
  
function operator+(a: Vec2, b: Vec2) -> Vec2:  
    return Vec2(a.x + b.x, a.y + b.y)  
  
function operator==(a: Vec2, b: Vec2) -> bool:  
    return a.x == b.x and a.y == b.y
```

```
v1 = Vec2(1, 2)  
v2 = Vec2(3, 4)  
v3 = v1 + v2  
print(v3.x)          # 4  
print(v1 == v2)      # false
```

一元运算符

接受一个参数。

```
function operator-(v: Vec2) -> Vec2:  
    return Vec2(-v.x, -v.y)
```

支持的运算符

类别	运算符
算术	+, -, *, /, %, **, //
比较	==, !=, <, <=, >, >=
位运算	&, \, ^, ~, <<, >>, >>>
逻辑	and, or, not
成员	in
索引访问	[]
索引赋值	[] =
函数调用	()
类型转换	as
复合赋值	+=, -=, *=, /=, %=, //=, **=, &=, \ =, ^=, <<=, >>=

为什么使用运算符重载？它给予领域类型自然的语法。Vec2 + Vec2 比 vec2_add(a, b) 更易读，== 让你的类型与 case 和比较无缝配合。

练习

1. **ADT**：定义一个 Animal 枚举，包含变体 Dog(name: str)、Cat(name: str, indoor: bool) 和 Fish。编写一个 describe(a: Animal) -> str 函数，使用 case 为每个变体返回描述。
2. 运算符重载：定义一个 Money record，包含 amount: int 和 currency: str。重载+，使得相同货币的两个 Money 值相加返回金额之和的新 Money。

[<- 上一篇：函数](#) | [下一篇：集合与迭代器 ->](#)

[English](#) | [日本語](#) | [简体中文](#)

集合与迭代器

[<- 上一篇：Record 与枚举](#) | [下一篇：错误处理 ->](#)

Ry 有四种集合类型：元组、列表、映射、集合。

元组

元组是将多个值组合在一起的不可变数据结构，可以持有不同类型的元素。

创建

```
t = (1, 3.14)
```

类型标注

```
t: (int, float) = (1, 3.14)
```

元素访问

使用 .0、.1 等索引来访问元素。

```
t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

作为函数返回值

想要返回多个值时，元组很方便。

```
function swap(a: int, b: int) -> (int, int):  
    return (b, a)
```

```
result = swap(1, 2)  
print(result.0)  # 2  
print(result.1)  # 1
```

限制

- 超出范围的索引（例如：对只有 2 个元素的元组使用 .2 访问）会产生编译错误。
 - 将元组直接传给 print 会产生错误。请逐个传递各元素。
-

列表

列表是由相同类型的元素组成的可变长度数据结构。

创建

```
xs = [1, 2, 3]
```

类型标注

```
xs: List<int> = [1, 2, 3]
```

索引访问

```
print(xs[0])  # 1
```

```
i = 1  
print(xs[i])  # 2
```

索引赋值

```
xs[0] = 99
```

length

```
print(length(xs))  # 3
```

print

```
print(xs)  # [1, 2, 3]
```

for 遍历

```
for x in xs:  
    print(x)
```

函数参数

```
function first(xs: List<int>) -> int:  
    return xs[0]
```

filter、map、sort

列表支持 filter、map、sort 操作。这些操作返回新列表，不会修改原始列表。

```
xs = [1, 2, 3, 4, 5]
```

```
# filter: 保留符合条件的元素
```

```
greater_than_three = filter(xs, (x: int) => x > 3)
print(greater_than_three)    # [4, 5]
```

```
# map: 转换每个元素
```

```
doubled = map(xs, (x: int) => x * 2)
print(doubled)              # [2, 4, 6, 8, 10]
```

```
# sort: 升序排序 (默认)
```

```
sorted = sort([3, 1, 2])
print(sorted)               # [1, 2, 3]
```

```
# 使用 UFCS (统一函数调用语法) 链式调用
```

```
# x.f(args) 等价于 f(x, args)
```

```
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)               # [20, 30, 40, 50]
```

reduce、fold

reduce 从第一个元素开始将列表归约为单一值。fold 则可提供明确的初始值。

```
xs = [1, 2, 3, 4, 5]
```

```
# reduce: 从第一个元素开始
```

```
total = reduce(xs, (a: int, b: int) => a + b)
print(total)                 # 15
```

```
# fold: 提供明确的初始值
```

```
total2 = fold(xs, 0, (a: int, b: int) => a + b)
print(total2)                # 15
```

any、all

any 如果至少有一个元素满足谓词则返回 true。all 如果所有元素都满足则返回 true。

```
xs = [1, 2, 3, 4, 5]
```

```
print(any(xs, (x: int) => x > 4))    # true
print(any(xs, (x: int) => x > 9))    # false
```

```
print(all(xs, (x: int) => x > 0))    # true
print(all(xs, (x: int) => x > 3))    # false
```

sum、min、max

```
xs = [3, 1, 4, 1, 5]
print(sum(xs))               # 14
print(min(xs))               # 1
print(max(xs))               # 5
```

first、last、is_empty

```
xs = [10, 20, 30]
print(first(xs))      # Some(10)
print(last(xs))       # Some(30)
print(is_empty(xs))   # false
```

enumerate、zip

enumerate 为每个元素附上索引。zip 将两个列表逐元素合并。

```
xs = [10, 20, 30]
indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

ys = ["a", "b", "c"]
zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]
```

限制

- 所有元素必须是相同类型，不能混合不同类型。
 - 空列表 [] 会产生错误。
 - 超出范围的访问会产生运行时错误 (exit(1))。
 - 元素类型支持 int、float、bool、str。
-

映射

映射是管理键值对的关联数组。

创建

```
m = {"a": 1, "b": 2}
```

类型标注

```
m: Map<str, int> = {"a": 1, "b": 2}
```

键访问

```
print(m["a"])    # 1
```

插入 / 更新

对新的键赋值即为插入，对已有的键赋值即为更新。

```
m["c"] = 3      # 插入新条目
m["a"] = 99     # 更新已有条目
```

length

```
print(length(m))    # 3
```

print

```
print(m)    # {a: 99, b: 2, c: 3}
```

has_key

检查键是否存在。

```
print(has_key(m, "a"))    # true
```

keys、values

keys 返回所有键的列表。values 返回所有值的列表。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
print(values(m))  # [1, 2, 3]
```

函数参数

```
function get_val(m: Map<str, int>, k: str) -> int:
    return m[k]
```

限制

- 所有键必须是相同类型，所有值也必须是相同类型。
 - 空映射需要类型标注。
 - 访问不存在的键会产生运行时错误 (exit(1))。
-

集合

集合是持有相同类型元素且不重复的集合类型。

创建

```
s = {1, 2, 3}
```

类型标注

```
s: Set<int> = {1, 2, 3}
```

in 运算符

使用 in 运算符检查元素是否包含在集合中。

```
print(2 in s)    # true
print(5 in s)    # false
```

add / remove

```
add(s, 4)        # 添加元素
remove(s, 1)     # 移除元素
add(s, 2)        # 已存在，因此忽略
```

length / print

```
print(length(s)) # 3
print(s)         # {2, 3, 4}
```

for 遍历

```
for x in s:
    print(x)
```

空集合

空集合需要类型标注。

```
empty: Set<int> = {}
```

限制

- 所有元素必须是相同类型。
 - 元素类型支持 `int`、`float`、`bool`、`str`。
-

迭代器

迭代器提供一种惰性的方式来处理集合。迭代器不会在每个步骤中创建中间列表，而是通过管道逐个处理元素。

为什么使用惰性迭代器？当你直接在列表上链式调用 `filter` 和 `map` 时，每个步骤都会创建新的中间列表。使用迭代器，元素逐个流经整个管道——无需中间分配。当处理大型集合或只需要前几个结果（使用 `take`）时，这一点很重要。

创建和消费

对集合调用 `iter()` 获取迭代器，调用 `to_list()` 将结果物化为列表：

```
xs = [1, 2, 3]
ys = to_list(iter(xs))    # [1, 2, 3]
```

链式操作

可以链式调用 `filter`、`map` 和 `take` 来构建管道。这使用了你在[函数](#)中学到的 UFCS 链式调用风格：

```
result = to_list(take(map(filter(iter([1, 2, 3, 4, 5])), (x: int) => x > 2), (x: int) => x * 2), 2))
print(result)    # [6, 8]
```

UFCS 链式调用风格（等价）：

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()
print(result)    # [6, 8]
```

以下是一个更实际的例子——处理成绩列表：

```
scores = [85, 42, 93, 67, 78, 55, 91]
```

获取前 3 个及格分数 (>= 60)，翻倍作为奖励

```
top_bonus = to_list(take(map(filter(iter(scores), (s: int) => s >= 60), (s: int) => s * 2), 3))
print(top_bonus)    # [170, 186, 134]
```

使用 next() 手动迭代

next() 返回 Option —— 如果有下一个元素则为 Some(value)，迭代器耗尽时为 None。你将在[错误处理](#)中进一步了解 Option。

```
it = iter([10, 20])
print(next(it))    # Some(10)
print(next(it))    # Some(20)
print(next(it))    # None
```

For 循环

迭代器可以直接在 for 循环中使用：

```
for x in filter(iter([1, 2, 3]), (x: int) => x > 1):
    print(x)        # 2, 3
```

遍历映射和集合

映射产生键值元组。集合产生单独的元素：

```
for k, v in iter({"a": 1, "b": 2}):
    print(f"{k} = {v}")

for x in iter({10, 20, 30}):
    print(x)
```

常见错误

- 忘记 to_list()：迭代器管道本身不做任何事——它是惰性的。你必须用 to_list()、for 循环或 next() 来消费它。
 - 过早调用 to_list()：在 filter() 之前放置 to_list() 会违背惰性求值的目的，因为它会先物化所有元素。
-

练习

1. 迭代器管道：给定 xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]，使用迭代器管道计算偶数之和。（提示：使用 .filter() 然后 .to_list() 和 sum()。）
 2. 手动迭代：对 [100, 200, 300] 创建迭代器，使用 case 区块处理 next() 返回的 Some 和 None 情况。
-

[<- 上一篇：Record 与枚举](#) | [下一篇：错误处理 ->](#)

[English](#) | [日本語](#) | [简体中文](#)

错误处理

[<- 上一篇：集合与迭代器](#) | [下一篇：包 ->](#)

Ry 提供三种互补的策略来处理错误和缺失值：**Option**（值可能缺失）、**Result**（操作可能失败）以及契约式设计（在边界处防止无效状态）。本教程涵盖这三种策略及其适用场景。

Option 类型

`Option<T>` 表示一个可能存在也可能不存在的值。它有两个变体：`Some(value)` 和 `None`。

```
x: Option<int> = Some(42)
print(x)      # Some(42)

y: Option<int> = None
print(y)      # None
```

提取值

使用 `case` 安全地提取内部值并处理 `None` 情况。这使用了你在[控制流](#)中学到的模式匹配：

```
case x:
  Some(v):
    print(v)      # 42
  None:
    print("nothing")
```

为什么使用 **Option**？它在类型系统中明确表示了值可能缺失的可能性。调用者必须处理 `None` 情况，而不是返回像 `-1` 这样的“魔术值”或检查 `null` ——编译器会确保这一点。

Option 的使用场景

你已经见过 `Option` 的实际使用：`iterator.next()` 返回 `Option<T>`，对每个元素给出 `Some(element)`，迭代器耗尽时给出 `None`。

Result 类型

`Result<T, E>` 用于可能失败的操作。成功时返回 `Ok(value)`，失败时返回 `Err(error)`。

```
function divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)
```

使用 case 处理 Result

```
r = divide(10, 0)
case r:
  Ok(v):
    print(v)
  Err(e):
    print(e.message)    # division by zero
```

? 运算符（错误传播）

当从一个返回 `Result` 的函数中调用另一个也返回 `Result` 的函数时，可以使用 `?` 自动传播错误。如果值是 `Ok`，它会被解包；如果是 `Err`，函数会立即返回该错误。

```
function safe_divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)

function divide_and_add(a: int, b: int) -> Result<int, Error>:
```

```
v = safe_divide(a, b)?    # 如果 b == 0 则提前返回 Err
return Ok(v + 1)
```

这等价于：

```
function divide_and_add(a: int, b: int) -> Result<int, Error>:
    case safe_divide(a, b):
        Ok(v):
            return Ok(v + 1)
        Err(e):
            return Err(e)
```

？运算符消除了样板代码，让你可以简洁地链式调用多个可能失败的操作：

```
function compute(a: int, b: int, c: int) -> Result<int, Error>:
    x = safe_divide(a, b)?
    y = safe_divide(x, c)?
    return Ok(y + 1)
```

使用 `and_then` 和 `map` 进行方法链

当你需要链接多个返回 `Result` 的操作但无法使用？（例如，不在返回 `Result` 的函数内部）时，可以使用 `and_then` 和 `map` 来避免深层嵌套的 `case` 语句。

`and_then` —— 链接本身返回 `Result` 的操作：

```
# 不需要嵌套 3 层 case:
result = safe_divide(100, 2)
    .and_then((v: int) => safe_divide(v, 5))
    .and_then((v: int) => safe_divide(v, 2))

case result:
    Ok(v): print(v)      # 5
    Err(e): print(e.message)
```

`map` —— 转换 `Ok` 值而不改变 `Result` 包装：

```
result = safe_divide(10, 2)
    .map((v: int) => v * 10)

case result:
    Ok(v): print(v)      # 50
    Err(e): print(e.message)
```

两个方法都会在 `Err` 时短路 —— 如果链中的任何步骤失败，错误会传播而不执行后续的闭包。

你可以在单个链中混合使用 `and_then` 和 `map`：

```
result = safe_divide(100, 10)
    .and_then((v: int) => safe_divide(v, 2))
    .map((v: int) => v + 100)
# Ok(105)
```

为什么使用 **Result**？它使错误处理变得显式，无需异常机制。类型签名准确告诉你哪些函数可能失败，而？运算符保持代码简洁。

常见错误：在不返回 `Result` 的函数中使用？会导致编译错误。？运算符只能在返回类型为 `Result` 的函数中使用。

契约式设计

Ry 支持 Eiffel 风格的契约式设计，包括前置条件 (`require`)、后置条件 (`ensure`) 和 record 不变式 (`invariant`)。Option 和 Result 在运行时处理错误，而契约从一开始就防止无效状态的出现。

完整规格请参阅[契约式设计参考手册](#)。

前置条件 (`require`)

使用 `require` 指定函数被调用时必须满足的条件：

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  new_balance: int = balance + amount
  return new_balance
```

如果任何前置条件失败，程序将终止并显示：

```
Contract violation: require failed in deposit()
```

后置条件 (`ensure`)

使用 `ensure` 指定函数返回时必须满足的条件。返回值绑定到用户选择的变量名：

```
function abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

由于 Ry 中函数参数是不可变的，你可以在 `ensure` 块中直接引用它们：

```
function increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

对于元组返回值，使用多个变量名：

```
function divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

组合 `require` 和 `ensure`

两者可以同时使用。require 必须在 ensure 之前：

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

Record 不变式 (invariant)

使用 `invariant` 指定 `record` 必须始终满足的条件。不变式在构造后以及每次字段赋值后都会检查：

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1          # Contract violation: invariant failed
```

为什么使用契约？它们直接在代码中记录和强制执行假设。如果函数要求 `amount > 0`，契约会在调用点立即捕获违规——而不是在函数体内部深处出现症状。

契约规则

- `require` 和 `ensure` 块是可选的，出现在函数体之前。
- 当两者同时存在时，`require` 必须在 `ensure` 之前。
- `ensure` 需要一个变量绑定（例如 `ensure v:`）来命名返回值。
- `invariant` 出现在 `record` 定义的末尾，在所有字段声明之后。
- 所有契约违规都会以 `exit(1)` 终止。

使用场景选择

策略	适用场景	示例
Option	值可能合理地缺失	查找键、 <code>iterator.next()</code>
Result	操作可能因有意义的错误而失败	文件 I/O、解析、网络调用
契约	无效输入不应该发生（程序员错误）	负数存款、越界索引

经验法则：- 对于可能因外部因素（用户输入、文件系统、网络）失败的操作，使用 **Result**。- 当“无值”是正常的、预期的结果时，使用 **Option**。- 使用契约尽早捕获程序员错误——它们是断言，不是错误处理。

常见错误

1. 忽略 **Result**：如果调用返回 `Result` 的函数却不处理它，你会丢失错误信息。
2. 在非 **Result** 函数中使用 `??` 运算符要求包含它的函数返回 `Result`。
3. 混淆 **Option** 和 **Result**：`Option` 有 `Some/None`；`Result` 有 `Ok/Err`。它们用途不同。

练习

1. **Result** 与 `??`：编写一个函数 `parse_and_double(s: str) -> Result<int, Error>`，使用辅助函数将字符串解析为整数并将其翻倍。使用 `??` 进行错误传播。
2. 契约：编写一个函数 `withdraw(amount: int, balance: int) -> int`，使用 `require` 确保 `amount > 0` 且 `amount <= balance`，并使用 `ensure` 确保结果为非负数。
3. **Option** 处理：编写一个函数，接受 `List<int>` 并返回第一个偶数作为 `Option<int>`，如果没有偶数则返回 `None`。

[<- 上一篇：集合与迭代器](#) | [下一篇：包](#) ->

[English](#) | [日本語](#) | [简体中文](#)

包

[<- 上一篇：错误处理](#) | [下一篇：并发](#) ->

Ry 使用包系统来管理跨文件和目录的代码。详细规格请参阅[包参考手册](#)。

from/import 语法

使用 from 语法导入其他文件的函数。

```
from math import sqrt, PI    # 选择性导入
from math                    # 全部导入（所有定义）
```

这样就可以使用 math.py 中定义的函数。

子目录（点分隔路径）

使用点分隔路径指定子目录中的包。

```
from utils.calc import add    # 从 utils/calc.py 导入
```

每个点对应一层目录分隔。

目录包

包可以是单一的 .py 文件，也可以是包含多个 .py 文件的目录。当包解析为目录时，其中所有的 .py 文件会自动加载。

```
mypackage/
  calc.py      # function add(), function sub()
  string.py    # function concat()

from mypackage          # 导入 add、sub、concat
from mypackage import add # 仅导入 add
```

不需要特殊的入口文件（如 __init__.py）。以 _ 开头的文件会被排除。

相对导入

使用前导 . 相对于当前文件所在目录进行导入。这对需要从同级模块导入的测试文件特别有用。

```
from .helper import greet    # 从同目录的 helper.py 导入
from .utils import add       # 从 utils/ 子目录导入
from .utils.calc import mul   # 从 utils/calc/ 嵌套子目录导入
from . import add, sub        # 从当前目录包导入符号
```

相对导入仅相对于当前文件所在目录解析——不会搜索标准库和其他搜索路径。这可以防止当你的项目中存在与标准库包同名的模块时发生名称冲突。

```
# 如果你的项目有 src/math/stats.ry:  
from .math import mean    # 始终解析为你的本地 math 包  
from math import sqrt     # 解析为标准库的 math 包
```

注意：不支持父目录导入（from ..）。

标准库 (std)

std 包会自动导入到所有程序中。不需要编写 from std —— 它始终可用。

```
# 这些函数无需导入即可使用  
print("hello")  
n = length("world")  
xs = range(5)
```

也可以从标准库的包中显式导入特定定义：

```
from str import contains
```

RY_HOME

标准库安装在 \$RY_HOME/share/std/。RY_HOME 的默认值为 ~/.ry。

```
export RY_HOME="$HOME/.ry"    # 默认
```

搜索路径优先级

包文件按以下顺序搜索：

1. 导入源文件的目录 —— 首先搜索包含导入语句的文件所在的目录。
 2. \$RY_HOME/share —— 标准库的位置。对于旧版安装，会回退至 \$RY_HOME/lib。
 3. 可执行文件相对的 share/ —— 相对于 ry 可执行文件的目录。对于旧版布局，会回退至可执行文件相对的 lib/。
 4. RY_PATH 环境变量 —— 如果未找到，按顺序搜索 RY_PATH 中指定的目录。
-

RY_PATH 环境变量

可以使用冒号分隔指定多个目录。

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

设置后，可以从任何地方导入指定目录中的包。

限制

- from 语句只能写在文件的顶层，不能写在函数或块内部。
- 多次导入同一包时，会自动跳过（不会发生重复导入）。
- 循环导入（A 导入 B，B 导入 A）会产生错误。

```
# 错误示例：a.ry 和 b.ry 互相导入  
# a.ry: from b import foo  
# b.ry: from a import bar <- 循环导入错误
```

[<- 上一篇：错误处理](#) | [下一篇：并发 ->](#)

[English](#) | [日本語](#) | [简体中文](#)

并发

[<- 上一篇：包](#) | [下一篇：测试 ->](#)

Ry 提供三种并发和并行执行模型：`async/await` 用于轻量级 I/O 密集型任务，`@parallel` 用于数据并行循环，`OS` 线程用于需要细粒度控制的 CPU 密集型工作负载。

async/await

`Task<T>` 是并发工作的运行时句柄。使用 `async function` 声明返回任务的函数，在另一个 `async function` 中使用 `await` 等待任务完成，从同步上下文使用 `block_on(task)`。

定义异步函数

```
async function add(a: int, b: int) -> int:
    return a + b
```

调用 `async function` 会立即返回 `Task<T>` —— 工作在后台开始：

```
t: Task<int> = add(20, 22)
```

等待结果

从同步代码中，使用 `block_on()`：

```
print(block_on(t))           # 42
print(block_on(add(1, 2)))    # 3
```

从异步代码中，使用 `await`：

```
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

```
print(block_on(double_add(3, 4))) # 14
```

组合异步函数

你可以自然地链接异步操作：

```
async function fetch_score() -> int:
    return 42
```

```
async function process() -> str:
    score = await fetch_score()
    return f"Score: {score * 2}"
```

```
print(block_on(process())) # Score: 84
```

为什么使用 `async/await`？它让你编写看起来像顺序代码的并发代码。运行时在线程池上高效调度任务 —— 非常适合大部分时间都在等待的 I/O 密集型工作。

常见错误：在非 `async` 函数中使用 `await` 会导致编译错误。从同步上下文调用时，请改用 `block_on()`。

@parallel

@parallel 指令将计数 for 循环并行化到多个 CPU 核心：

```
@parallel
for i in range(8):
    print(i)
```

限制

@parallel for 设计用于独立的迭代。它拒绝：- **break** 和 **continue** ——并行迭代之间没有有意义的顺序。- 写入外部可变量 ——这会导致数据竞争。

仅支持计数循环（range(...) 或整数 .. 范围）。

为什么使用 @parallel？这是并行化独立工作的最简单方式。无需锁、无需原子操作 —— 只需添加指令，运行时会处理分发。

OS 线程

对于 CPU 密集型任务或需要细粒度同步时，使用 thread 模块创建 OS 线程。

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add
```

创建线程

thread_spawn 接受一个闭包并启动新的 OS 线程：

```
counter = atomic_int_new(0)

t = thread_spawn():
    atomic_int_add(counter, 1)
)

thread_join(t)
print(atomic_int_load(counter))    # 1
```

注意：捕获的变量会被复制到线程中。对于原始类型（int、str 等），这会产生一个独立的副本。对于不透明句柄类型（AtomicInt、Lock 等），底层资源是共享的 —— 这正是同步所需要的。

使用原子操作共享状态

AtomicInt 提供无锁的线程安全整数操作：

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)

t1 = thread_spawn():
    atomic_int_add(counter, 1)
)
t2 = thread_spawn():
    atomic_int_add(counter, 1)
)

thread_join(t1)
thread_join(t2)
print(atomic_int_load(counter))    # 2
```


使用锁进行互斥

当你需要保护临界区（不止单个原子操作）时，使用 Lock：

```
from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
from thread import atomic_int_new, atomic_int_load, atomic_int_add

lock = lock_new()
counter = atomic_int_new(0)

t = thread_spawn():
    lock_acquire(lock)
    # 临界区：同一时间只有一个线程
    atomic_int_add(counter, 1)
    lock_release(lock)
)

lock_acquire(lock)
atomic_int_add(counter, 1)
lock_release(lock)

thread_join(t)
print(atomic_int_load(counter))    # 2
```

其他同步原语

thread 模块还提供：

原语	使用场景
RWLock	多个读者或一个写者（读多写少的场景）
Semaphore	限制对资源的并发访问
Barrier	等待 N 个线程到达同一点
AtomicBool	线程安全的布尔标志

完整 API 请参阅[线程参考手册](#)。

网络（TCP 套接字）

Ry 透过 net 模组提供 TCP 套接字支援。网路操作返回 Result 类型（来自[错误处理](#)），因为连接可能失败。

核心 TCP 原语：

函数	说明
bind(host, port)	分配一个监听器。返回 Result<TcpListener, Error>
listen(listener, backlog)	开始接受连接。返回 Result<Unit, Error>
accept(listener)	等待下一个连接。返回 Result<TcpStream, Error>
connect(host, port)	打开一个出站流。返回 Result<TcpStream, Error>
send(stream, bytes)	发送 List<u8>。返回 Result<int, Error>
receive(stream, max)	读取最多 max 位元组。返回 Result<List<u8>, Error>
close(handle)	释放 TcpListener、TcpStream 或 TlsStream

以下是一个在 async 伺服器与同步客户端之间进行 echo 交换的范例：

```
from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str
```

```
async function echo_server(server: TcpListener) -> str:
    case accept(server):
        Ok(conn):
            case receive(conn, 4096):
                Ok(data):
                    case send(conn, data):
                        Ok(_):
                            ...
                        Err(e):
                            ...
                    Err(e):
                        ...
                close(conn)
            Err(e):
                ...
        close(server)
    return "done"
```

```
function run_client(port: int) -> str:
    case connect("127.0.0.1", port):
        Ok(conn):
            case send(conn, to_bytes("hello")):
                Ok(_):
                    ...
                Err(e):
                    ...
            case receive(conn, 4096):
                Ok(resp):
                    case bytes_to_str(resp):
                        Ok(msg):
                            close(conn)
                            return msg
                        Err(e):
                            ...
                Err(e):
                    ...
            close(conn)
        return "fail"
    Err(e):
        return "fail"
```

```
case bind("127.0.0.1", 0):
    Ok(server):
        case listen(server, 1):
            Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                print(run_client(port))    # hello
                block_on(t)
            Err(e):
```

```

        print(e.message)
    Err(e):
        print(e.message)

```

完整 TCP API 请参阅[网路参考手册](#)，其中包括 TLS (`tls_connect`) 与每个流的超时设定。

选择合适的模型

特性	async/await	@parallel	thread 模块
最适合	I/O 密集型任务	数据并行循环	CPU 密集型、细粒度控制
开销	低 (任务调度)	低 (自动)	较高 (OS 线程创建)
同步	通过 <code>await</code> 自动	不需要 (独立迭代)	手动 (Lock、Semaphore 等)
复杂度	中等	低	高

常见错误

1. 在 `async function` 外使用 `await`：从同步上下文应使用 `block_on()` 代替。
2. 在 `@parallel` 中写入外部变量：编译时会被拒绝以防止数据竞争。
3. 忘记 `thread_join`：如果主程序在线程完成前退出，线程的工作可能会丢失。
4. 在线程间共享原始变量：原始类型会被复制到闭包中。使用 `AtomicInt` 或 `Lock` 进行状态共享。

练习

1. `async/await`：编写两个 `async function` 函数——一个返回名字，一个返回姓氏。编写第三个 `async function`，`await` 两者并返回全名。使用 `block_on()` 打印结果。
2. 线程与原子操作：创建 5 个线程，每个线程向共享的 `AtomicInt` 加 10。在 `join` 所有线程后，验证计数器为 50。

[<- 上一篇：包](#) | [下一篇：测试](#) ->

[English](#) | [日本語](#) | [简体中文](#)

测试

[<- 上一篇：并发](#) | [下一篇：构建项目](#) ->

Ry 内建了使用 `@describe`、`@it`、`expect` 的 RSpec 风格测试语法。详细规格请参阅[测试参考手册](#)。

运行测试

```

ry test                # 自动发现并运行所有 *.test.ry 文件
ry test tests/spec     # 递归运行指定目录下所有 *.test.ry 文件
ry test tests/my_test.test.ry # 运行特定的测试文件
ry test -p             # 并行运行所有测试 (-p 或 --parallel)

```

所有测试通过时，退出码为 0；若有任一测试失败，则为 1。

不带参数运行时，`ry test` 会搜索 `package.toml` 来找到项目根目录，然后递归地发现所有 `*.test.ry` 文件。

编写测试

将 `@it` 附加到命名函数上以宣告一个测试用例，并将一组相关测试包裹在带有 `@describe` 注解的函数中。

```
@it("should add integers")
function test_add():
    expect(1 + 2).to_eq(3)

@describe("Calculator")
function calculator_tests():
    @it("should subtract integers")
    function test_sub():
        expect(5 - 3).to_eq(2)

    @it("should check booleans")
    function test_bool():
        expect(3 > 1).to_be_true()
```

- `@it` 接受一个描述字串。被装饰的函数成为测试主体
- `@describe` 会将其函数主体中定义的内部 `@it` 函数分组。可以巢状嵌套；输出按巢状深度比例缩排
- 直接在 `@describe` 函数主体中宣告的变数会作为共享设置 (**shared setup**)，并被每个内部 `@it` 函数捕获

```
@describe("shared setup")
function shared_setup_tests():
    base = 100
    offset = 5

    @it("should use base value")
    function test_base():
        expect(base).to_eq(100)

    @it("should use base and offset")
    function test_combined():
        expect(base + offset).to_eq(105)
```

- `expect`、`mock` 和 `verify` 仅可在 `ry test` 中使用（普通的 `ry` 执行会产生编译错误）

旧版 `lambda` 形式：带 `lambda` 参数的 `describe("name", (): ...)` 和 `it("desc", (): ...)` 仍然可解析，但已弃用。新代码请优先使用作用于命名函数的指令形式。

匹配器

匹配器	说明	支持类型
<code>to_eq(expected)</code>	等值比较	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_not_eq(expected)</code>	不等值断言	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_be_true()</code>	<code>true</code> 断言	<code>bool</code>
<code>to_be_false()</code>	<code>false</code> 断言	<code>bool</code>
<code>to_be_none()</code>	<code>None</code> 断言	<code>Option</code>
<code>to_be_some()</code>	<code>Option</code> 为 <code>Some</code> 的断言	<code>Option</code>
<code>to_be_ok()</code>	<code>Result</code> 为 <code>Ok</code> 的断言	<code>Result</code>
<code>to_be_err()</code>	<code>Result</code> 为 <code>Err</code> 的断言	<code>Result</code>
<code>to_contain(val)</code>	容器包含值的断言	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>

匹配器	说明	支持类型
<code>to_not_contain(val)</code>	容器不包含值的断言	List, Set, Map, str
<code>to_be_greater_than(v)</code>	<code>actual > v</code> 断言	int, float
<code>to_be_less_than(v)</code>	<code>actual < v</code> 断言	int, float
<code>to_be_greater_than_or_eq(v)</code>	<code>actual >= v</code> 断言	int, float
<code>to_be_less_than_or_eq(v)</code>	<code>actual <= v</code> 断言	int, float
<code>to_have_length(n)</code>	长度等于 <code>n</code> 的断言	List, Set, Map, str
<code>to_be_empty()</code>	长度为 0 的断言	List, Set, Map, str
<code>to_start_with(prefix)</code>	字符串以前缀开头的断言	str
<code>to_end_with(suffix)</code>	字符串以后缀结尾的断言	str

fail

`fail()` 立即将当前测试标记为失败。

```
@it("should handle error")
function test_should_handle_error():
  case result:
    Ok(v):
      fail("expected error")
    Err(e):
      expect(e.message).to_eq("not found")
```

- `fail()` —— 以通用消息标记测试失败
- `fail(message)` —— 以自定义消息标记测试失败
- 仅在 `ry test` 模式下可用

输出格式

```
Calculator
+ should add integers
+ should subtract integers
- should check booleans
  line 10: expected true, got false
```

2 passed, 1 failed

+ 表示通过（绿色），- 表示失败（红色）。巢状的`@describe` 群组会将其内部测试依巢状深度按比例缩排：

```
outer group
  inner group
    + should pass deeply nested test
```

模拟 (Mock)

```
mock(fn_name, replacement)
```

在当前的 `@it` 块中将函数替换为模拟实现。`@it` 块结束时会自动恢复。原函数的 `require` 和 `ensure` 契约仍然会对模拟调用执行。

```
function fetch_data() -> str:
  return "real data"
```

```

@describe("mocking")
function mocking_tests():
  @it("should replace function")
  function test_replace():
    mock(fetch_data, () => "fake")
    expect(fetch_data()).to_eq("fake")

  @it("should auto-restore after it block")
  function test_restore():
    expect(fetch_data()).to_eq("real data")

verify(fn_name)

```

返回模拟函数被调用的次数。

```

@describe("verify")
function verify_tests():
  @it("should count calls")
  function test_count():
    mock(fetch_data, () => "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)

```

参数化测试

结合 @each 与 @it 作用于命名函数上，以多组输入运行同一个测试：

```

@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
@it("should add {0} + {1} = {2}")
function test_add_each(a: int, b: int, expected: int):
  expect(a + b).to_eq(expected)

```

每个元组成为一个独立的测试用例。描述中的 {0}、{1} 等会被替换为实际值。函数的参数列表必须与元组的元数匹配。

基于属性的测试

结合 @property 与 @it 作用于命名函数上，以随机生成的输入进行测试：

```

@property(count=100)
@it("should verify addition is commutative")
function test_add_commutative(a: int, b: int):
  expect(a + b).to_eq(b + a)

```

测试以随机值运行 count 次。Ry 会从每个参数的类型注解推断生成器。失败时会显示反例。

使用契约进行测试

契约（来自[错误处理](#)）与模拟协同工作：原函数的 require 和 ensure 契约仍然会对模拟调用执行。这意味着契约充当隐式测试断言。

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
  ensure v:
    v > balance
  return balance + amount

@describe("deposit")
function deposit_tests():
  @it("should enforce contracts even when mocked")
  function test_contract():
    mock(deposit, (amount: int, balance: int) => balance + amount)
    expect(deposit(10, 100)).to_eq(110)
    # deposit(-1, 100) 会以 "require failed" 终止
```

为什么这很重要：你可以模拟实现细节，同时保留契约安全网。如果模拟违反了后置条件，测试会立即捕获。

限制

- 巢状只支援作用于命名函数的 @describe。旧版 lambda 形式 describe("name", (): ...) 无法巢状
 - 不支援 before_each / after_each —— 请改用在 @describe 函数主体中宣告的共享设置变数
 - 重载函数及 @native 函数无法模拟
-

练习

1. 基本测试：为 max(a: int, b: int) -> int 函数编写 describe 块，覆盖相等值、正数和负数的情况。
 2. 模拟：编写 fetch_temperature() -> int 函数并返回一个值。在测试中模拟它返回固定值，使用 verify 检查它被调用了恰好一次。
 3. 参数化测试：使用 @each 测试 is_even(n: int) -> bool 函数，输入为 [(2, true), (3, false), (0, true), (-4, true)]。
-

[<- 上一篇：并发](#) | [下一篇：构建项目](#) ->

[English](#) | [日本語](#) | [简体中文](#)

构建项目

[<- 上一篇：测试](#)

在本教程中，你将构建一个**任务跟踪器**——一个管理内存中待办事项列表的小应用程序。这个项目将把你学过的大部分语言特性串联起来：

- **Record** 和 **ADT enum**（数据建模）
- 集合和迭代器（过滤和转换任务）
- 错误处理（Result、Option、契约）
- **F** 字符串和 **UFCS**（可读的输出和链式调用）
- 带默认参数的函数（灵活的 API）
- 模块（跨文件组织代码）
- 测试（验证行为）

项目设置

创建一个新项目：

```
ry new task-tracker
cd task-tracker
```

这会生成以下结构：

```
task-tracker/
  package.toml
  src/
    main.ry
```

步骤 1：定义数据模型

创建 `src/model.ry`，定义任务 `Record` 和状态 `enum`：

```
enum Status:
  Todo
  InProgress
  Done

record Task:
  id: int
  title: str
  status: Status
  invariant:
    length(title) > 0

function create_task(id: int, title: str, status: Status = Status::Todo) -> Task:
  require:
    length(title) > 0
  return Task(id, title, status)
```

这里同时使用了多个特性：- **ADT enum** 用于 `Status`（来自 [Record](#) 和 [Enum](#)）- **Record** 不变条件确保标题永远不为空（来自[错误处理](#)）- 默认参数 使 `status` 默认为 `Todo`（来自[函数](#)）- 契约（`require`）作为构造时的安全检查（来自[错误处理](#)）

步骤 2：任务操作

在 `src/model.ry` 中添加处理任务列表的函数：

```
function add_task(tasks: List<Task>, title: str) -> List<Task>:
  id = length(tasks) + 1
  task = create_task(id, title)
  append(tasks, task)
  return tasks

function find_task(tasks: List<Task>, id: int) -> Option<Task>:
  for t in tasks:
    if t.id == id:
      return Some(t)
```



```

    return None

function complete_task(tasks: List<Task>, id: int) -> Result<List<Task>, Error>:
    case find_task(tasks, id):
        Some(t):
            t.status = Status::Done
            return Ok(tasks)
        None:
            return Err(Error(f"task {id} not found"))

function pending_tasks(tasks: List<Task>) -> List<Task>:
    return tasks
        .iter()
        .filter((t: Task) => t.status == Status::Todo)
        .to_list()

```

注意以下模式：- **Option** 用于 `find_task` —任务可能不存在（来自[错误处理](#)）- **Result** 用于 `complete_task` —完成不存在的任务是一个错误 - 迭代器管道 与 UFCS 链式调用在 `pending_tasks` 中使用（来自[集合和函数](#)）- **F** 字符串用于错误消息（来自[变量和类型](#)）

步骤 3：显示

在 `src/model.ry` 中添加显示函数：

```

function format_task(t: Task) -> str:
    marker = "[ ]"
    case t.status:
        Status::Todo:
            marker = "[ ]"
        Status::InProgress:
            marker = "[~]"
        Status::Done:
            marker = "[x]"
    return f"{marker} {t.id}. {t.title}"

function print_tasks(tasks: List<Task>):
    if is_empty(tasks):
        print("No tasks.")
        return
    for t in tasks:
        print(format_task(t))

```

步骤 4：主程序

编辑 `src/main.ry`：

```

from model import create_task, add_task, complete_task, pending_tasks, print_tasks, Status, Task

tasks: List<Task> = []

# 添加一些任务
tasks = add_task(tasks, "Buy groceries")
tasks = add_task(tasks, "Write documentation")

```

```
tasks = add_task(tasks, "Review pull request")
```

```
print("All tasks:")
print_tasks(tasks)
```

完成一个任务

```
case complete_task(tasks, 1):
    Ok(updated):
        tasks = updated
    Err(e):
        print(f"Error: {e}")
```

```
print("\nPending tasks:")
print_tasks(pending_tasks(tasks))
```

运行程序：

```
ry src/main.ry
```

预期输出：

```
All tasks:
[ ] 1. Buy groceries
[ ] 2. Write documentation
[ ] 3. Review pull request
```

```
Pending tasks:
[ ] 2. Write documentation
[ ] 3. Review pull request
```

步骤 5：编写测试

创建 tests/model.test.ry：

```
from model import create_task, add_task, find_task, complete_task, pending_tasks, Status, Task
```

```
describe("Task model", ():
    it("creates a task with default status", ():
        t = create_task(1, "Test task")
        expect(t.status == Status::Todo).to_be_true()
    )

    it("adds tasks to a list", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "First")
        tasks = add_task(tasks, "Second")
        expect(length(tasks)).to_eq(2)
    )

    it("finds a task by id", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "Target")
        case find_task(tasks, 1):
            Some(t):
                expect(t.title).to_eq("Target")
```

```

        None:
            fail("expected to find task")
    )

    it("returns None for missing task", ():
        tasks: List<Task> = []
        expect(find_task(tasks, 99)).to_be_none()
    )

    it("completes a task", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "Do it")
        case complete_task(tasks, 1):
            Ok(updated):
                remaining = pending_tasks(updated)
                expect(is_empty(remaining)).to_be_true()
            Err(e):
                fail("unexpected error")
    )

    it("returns error for invalid id", ():
        tasks: List<Task> = []
        case complete_task(tasks, 999):
            Ok(_):
                fail("expected error")
            Err(e):
                expect(e.message != "").to_be_true()
    )
)

```

运行测试：

```
ry test
```

下一步

以下是一些扩展此项目的方式：

- 添加截止日期 — 使用 `Option<str>` 添加可选的截止日期字段
- 持久化到 **JSON** — 使用 `json` 模块将任务保存到文件或从文件加载
- 添加优先级 — 将 `Status` 扩展为带有优先级级别的 ADT
- 并发处理 — 使用 `@parallel` 并行处理一批任务

你已经拥有了扎实的 `Ry` 基础。请查阅[参考手册](#)以了解完整的语言规格。

[<- 上一篇：测试](#)